WEEK 2 트랜스포머 핵심 구성 요소

2.1 위치 임베딩 (Positional Embedding) | 마야

트랜스포머는 입력 토큰을 한꺼번에 처리하므로, 위치 정보가 없으면 단어의 순서를 구별하기 어렵다. 예를 들어 "개가 사람을 문다"와 "사람이 개를 문다"는 같은 단어로 이루어져 있지만 의미가 다르다.

〈핵심 아이디어〉

- 각 토큰의 **토큰 임베딩**에 그 위치를 나타내는 **위치 임베딩**을 더해 순서 정보를 제공한다.
- 두 벡터는 차원이 같아야 하며, 모델은 결합된 벡터를 통해 단어의 의미와 순서를 함께 활용할 수 있다.

〈토큰과 임베딩의 흐름〉

1. 텍스트를 단어, 서브워드 또는 문장부호 등의 토큰으로 나누고 각 토큰에 ID를 부여한다.

2. 토큰 임베딩 테이블에서 ID에 대응하는 의미 벡터를 가져온다. (예: 768차원)

3. 위치 임베딩 테이블에서 위치 번호(0, 1, 2, …)에 대응하는 벡터를 가져온다.

4. 두 벡터를 더해 입력 표현을 만든다. BERT 계열에서는 이후 정규화와 드롭아웃 등을 적용한다.

〈출력 크기 및 위치 표현 방식〉

- **[1, 5, 768]** = 문장 1개, 토큰 5개, 토큰마다 768차원 벡터를 뜻한다.
- **학습형 절대 위치 임베딩** = 위치별 벡터를 모델 파라미터로 두고 사전 학습 과정에서 값을 조정하는 방식이다.
- **고정형 절대 위치 표현** = 사인·코사인 함수로 위치별 패턴을 만드는 방식이다.
- **상대 위치 표현** = 두 토큰의 거리나 상대적인 순서를 반영한다. DeBERTa가 대표적인 예다.
- **RoPE(Rotary Position Embedding)** = Q와 K에 위치별 회전을 적용해 상대적 위치 정보를 어텐션 계산에 반영한다. GPT-NeoX 등이 활용한다.

〈한 문장 정리〉

토큰은 의미 벡터로, 위치는 위치 벡터로 표현하여 트랜스포머가 문장 속 단어의 의미와 순서를 함께 처리하도록 한다.

2.2 어텐션 메커니즘 (Attention Mechanism) | 박근영

〈셀프 어텐션 (Self-Attention)〉

셀프 어텐션은 같은 시퀀스 안에서 각 토큰이 다른 토큰들과 얼마나 관련 있는지 계산하고, 중요도에 따라 정보를 결합해 문맥을 반영한 임베딩(Contextualized Embedding)을 만든다.

- **동작 흐름** = 토큰 간 관련성 계산 → 어텐션 가중치 부여 → Softmax로 정규화 → 가중합으로 새로운 임베딩 생성
- **예시** = "눈이 내려 길이 하얗게 변했다"와 "눈이 아파 안과에 갔다"에서 같은 "눈"이라도 주변 단어를 참고해 서로 다른 의미를 반영한다.

〈스케일드 점곱 어텐션 (Scaled Dot-Product Attention)〉

셀프 어텐션을 계산하는 대표적인 방식이다. Query와 Key의 점곱으로 관련성을 구하고, 그 결과를 이용해 Value를 가중합한다.

1. **Q, K, V 생성** = 토큰 표현에 서로 다른 선형 변환을 적용한다. Query는 찾는 정보, Key는 비교할 특징, Value는 전달할 정보에 대응한다.

2. **어텐션 점수 계산** = Query와 Key를 점곱해 토큰 간 관련성 점수를 만든다.

3. **스케일링과 정규화** = 점수를 키 벡터 차원의 제곱근으로 나누고 Softmax를 적용한다. 각 Query에 대한 가중치 합은 1이 된다.

4. **문맥 표현 생성** = 가중치를 Value에 적용해 문맥이 반영된 새로운 벡터를 만든다.

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})\, V$$

- QK^T: 관련성 점수 $\sqrt{d_k}$: 점수 스케일링 **Softmax**: 가중치 정규화 **V**: 가중합 대상

〈멀티 헤드 어텐션 (Multi-Head Attention)〉

여러 어텐션 헤드를 병렬로 사용해 토큰 사이의 다양한 관계와 특징을 학습하는 방식이다.

1. 서로 다른 선형 변환으로 각 헤드의 Q, K, V를 만든다.

2. 각 헤드에서 어텐션을 독립적으로 계산한다.

3. 헤드별 결과를 연결(Concatenate)한 뒤 최종 선형 변환을 적용한다.

〈핵심 정리〉

- **셀프 어텐션**: 같은 문장 안의 토큰 간 관계를 반영하는 메커니즘
- **스케일드 점곱 어텐션**: Q, K, V를 이용해 관련성 점수와 가중합을 계산하는 방법
- **멀티 헤드 어텐션**: 여러 어텐션을 병렬로 수행해 다양한 문맥을 반영하는 구조

2.3 피드 포워드 층 (Feed-Forward Layer) | 신민영

트랜스포머의 인코더와 디코더 블록에서는 어텐션 이후 피드 포워드 층을 사용한다. 어텐션이 다른 토큰에서 정보를 모은다면, 피드 포워드 층은 각 토큰의 표현을 개별적으로 가공한다.

〈주요 개념〉

- **피드 포워드 층(FFN)** = 토큰 벡터를 더 큰 차원으로 확장했다가 원래 차원으로 줄이는 두 개의 선형 층과 비선형 활성화 함수로 구성된다.
- **위치별 피드 포워드(Position-wise FFN)** = 모든 토큰 위치에 같은 가중치를 적용하지만, 이 단계 자체에서 토큰끼리 정보를 교환하지 않는다.
- **GELU** = 선형 변환 사이에 비선형성을 추가해 복잡한 패턴을 표현하도록 돕는 활성화 함수다.
- **드롭아웃(Dropout)** = 학습 중 일부 활성값을 무작위로 비활성화해 과적합을 줄이는 기법이다.

〈동작 과정: 펼치기 → 활성화 → 접기〉

예: BERT 768차원 → 3072차원 → GELU → 768차원

- 첫 번째 선형 층에서 벡터를 확장하고, 활성화 함수를 거쳐 두 번째 선형 층에서 다시 원래 차원으로 변환한다.
- 전통적인 트랜스포머 계열에서는 중간 차원을 hidden dimension의 약 4배로 설정하는 경우가 많다.

〈셀프 어텐션과의 차이〉

- **셀프 어텐션** = 여러 토큰이 서로 정보를 주고받는 단계 (함께 회의)
- **피드 포워드** = 각 토큰이 모은 문맥 정보를 자신의 벡터 안에서 가공하는 단계 (각자 정리)
- **예시** = "나는 배를 먹었다"에서 어텐션은 "배"가 "먹었다"를 참고하도록 하고, FFN은 반영된 정보를 비선형 변환으로 가공한다.

〈특징 및 블록 내부 흐름〉

- 일반적인 BERT형 인코더 블록에서는 FFN이 큰 비중의 파라미터를 차지하며, 모델의 표현 능력에 중요한 역할을 한다.
- 같은 변환을 각 위치에 독립적으로 적용한다는 점에서 커널 크기 1인 1D 합성곱과 유사하다.
- **흐름** = 입력 → 셀프 어텐션 → 피드 포워드 → 다음 블록 또는 출력 (실제 블록에는 잔차 연결과 정규화도 포함)

2.4 Transformer Head | 박세홍

〈Head의 역할과 기본 구조〉

Transformer는 입력을 처리해 각 토큰의 문맥 벡터(Context Vector)를 생성한다. Head는 이러한 벡터를 실제 작업에 필요한 출력으로 변환하는 층이다.

입력 → Transformer → Context Vector → Head → 최종 출력

- **기본 연산** = z = Wh + b (h: 문맥 벡터, W: 학습 가중치 행렬, b: 편향, z: logit)
- W는 하나의 숫자가 아니라 클래스 또는 어휘 항목에 대응하는 여러 가중치 벡터로 구성된 행렬이다.

〈Classification Head〉

- 문맥 벡터를 클래스별 점수로 바꾸며, 기본 연산은 $z = W_{CL}h + b$ 이다.
- W의 각 행이 한 클래스에 대응하고, 입력 벡터와의 내적으로 클래스별 logit을 계산한다.
- 단일 라벨 다중 클래스 분류에서는 Softmax로 클래스별 확률을 얻는다. 다중 라벨 분류에서는 일반적으로 클래스별 Sigmoid를 사용한다.

〈LM Head〉

- 문맥 벡터를 전체 어휘(Vocabulary)의 토큰별 점수로 바꾸며, 기본 연산은 $z = W_{LM}h + b$ 이다.
- W의 각 행이 어휘의 한 토큰에 대응한다. Softmax를 적용하면 해당 위치에서 예측할 토큰의 확률 분포를 얻는다.
- Causal LM은 다음 토큰을, Masked LM은 가려진 토큰을 예측하는 등 학습 목표에 따라 정답 토큰이 달라진다.

〈같은 W를 여러 위치에 사용할 수 있는 이유〉

- Transformer는 토큰 위치가 달라도 같은 hidden dimension의 벡터를 출력한다.
- 그래서 동일한 작업의 Head는 각 위치에서 같은 가중치 W를 사용해 표현을 클래스 또는 어휘 점수로 변환할 수 있다.

〈Head의 학습〉

- **LM Head** = 문장에서 예측 목표를 구성하고 모델의 예측과 비교해 Loss를 계산한다. 정답을 텍스트 자체에서 얻으므로 자기지도학습(Self-Supervised Learning)이 가능하다.
- **Classification Head** = 보통 외부에서 제공된 label을 정답으로 사용하며, 일반적으로 지도학습(Supervised Learning)에 해당한다.

〈Loss와 역전파〉

- Head 자체가 Loss를 계산하는 것은 아니다. **Transformer → Head → Logits → 정답과 비교 → Loss** 순서로 계산된다.
- 역전파는 **Loss → Head → Transformer** 방향으로 이루어져, 파인튜닝 시 Head뿐 아니라 Transformer 내부 가중치도 함께 업데이트될 수 있다.